

Process Synchronization

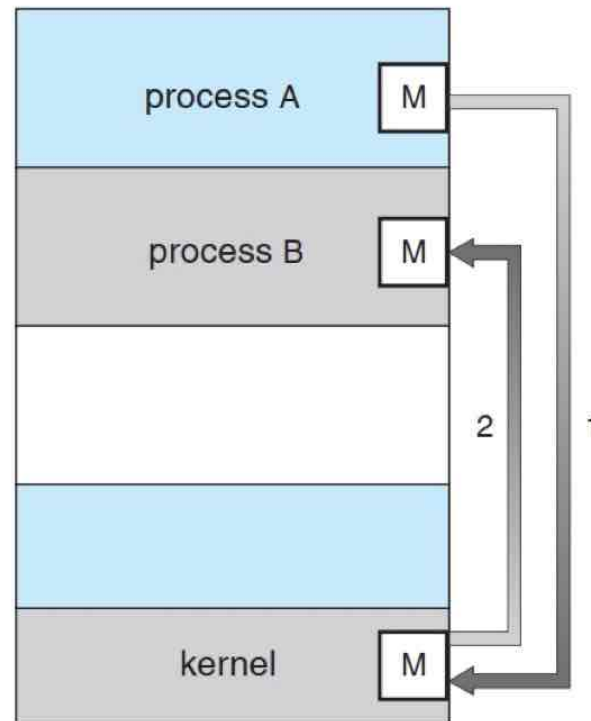
Inter-Process Communication (IPC)

- Processes within a system may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **inter-process communication (IPC)**

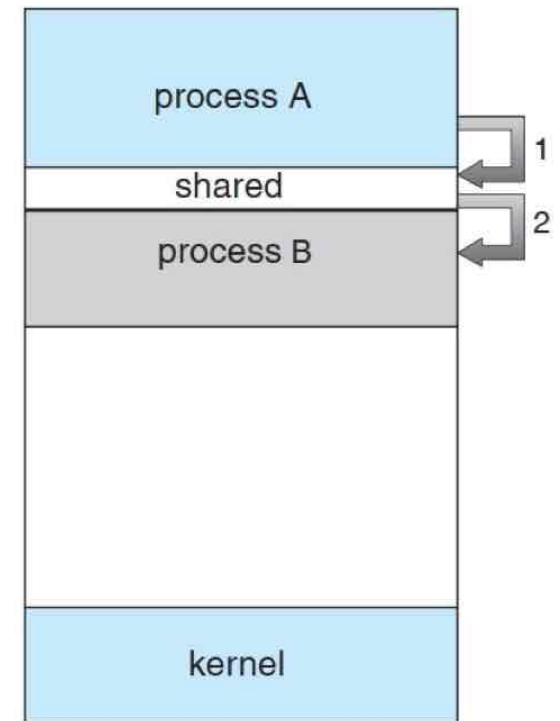
Communications Models

Two models of IPC

- a. Message passing
- b. Shared memory



(a)



(b)

Message Passing

- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - **send**(*message*)
 - **receive**(*message*)
- The message size is either fixed or variable
- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive

Message Passing (Cont'd)

- Implementation of communication link

- Physical:

- Shared memory

- Hardware bus

- Network

- Logical:

- Direct or indirect

- Synchronous or asynchronous

- Automatic or explicit buffering

Message Passing (Cont'd)

- Direct Communication:

- Processes must name each other explicitly:
 - send (P, message) – send a message to process P
 - receive(Q, message) – receive a message from process Q

- Indirect Communication:

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox

Message Passing (Cont'd)

- Synchronous Communication:
 - Blocking is considered synchronous
 - Blocking send -- the sender is blocked until the message is received
 - Blocking receive -- the receiver is blocked until a message is available
- Asynchronous Communication:
 - Non-blocking is considered asynchronous
 - Non-blocking send -- the sender sends the message and continue
 - Non-blocking receive -- the receiver receives: A valid message, or Null message

Message Passing (Cont'd)

- Buffering:

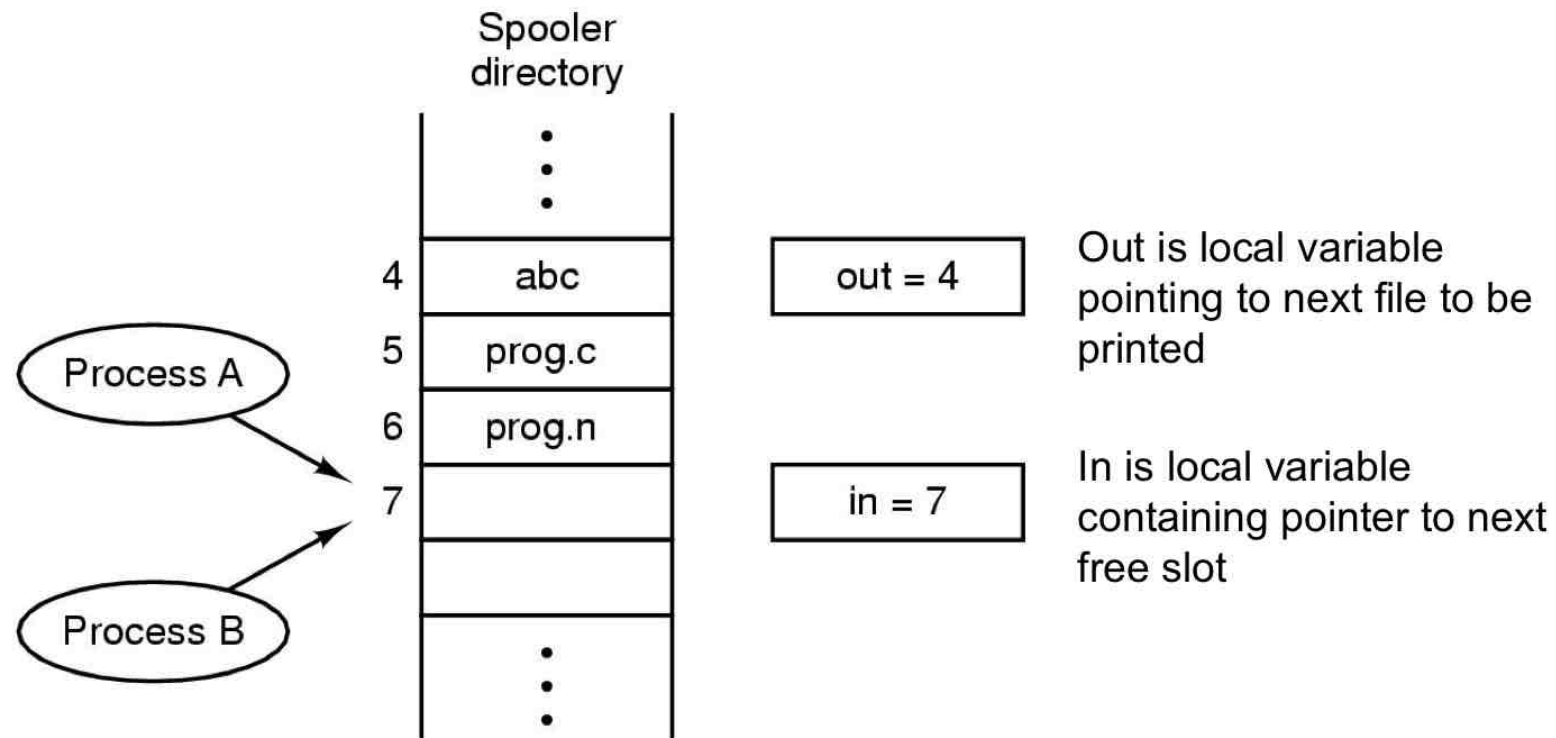
- Queue of messages attached to the link.
- Implemented in one of three ways
 - Zero capacity – no messages are queued on a link.
Sender must wait for receiver (rendezvous)
 - Bounded capacity – finite length of n messages
Sender must wait if link full
 - Unbounded capacity – infinite length
Sender never waits

Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues are to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

Race Conditions

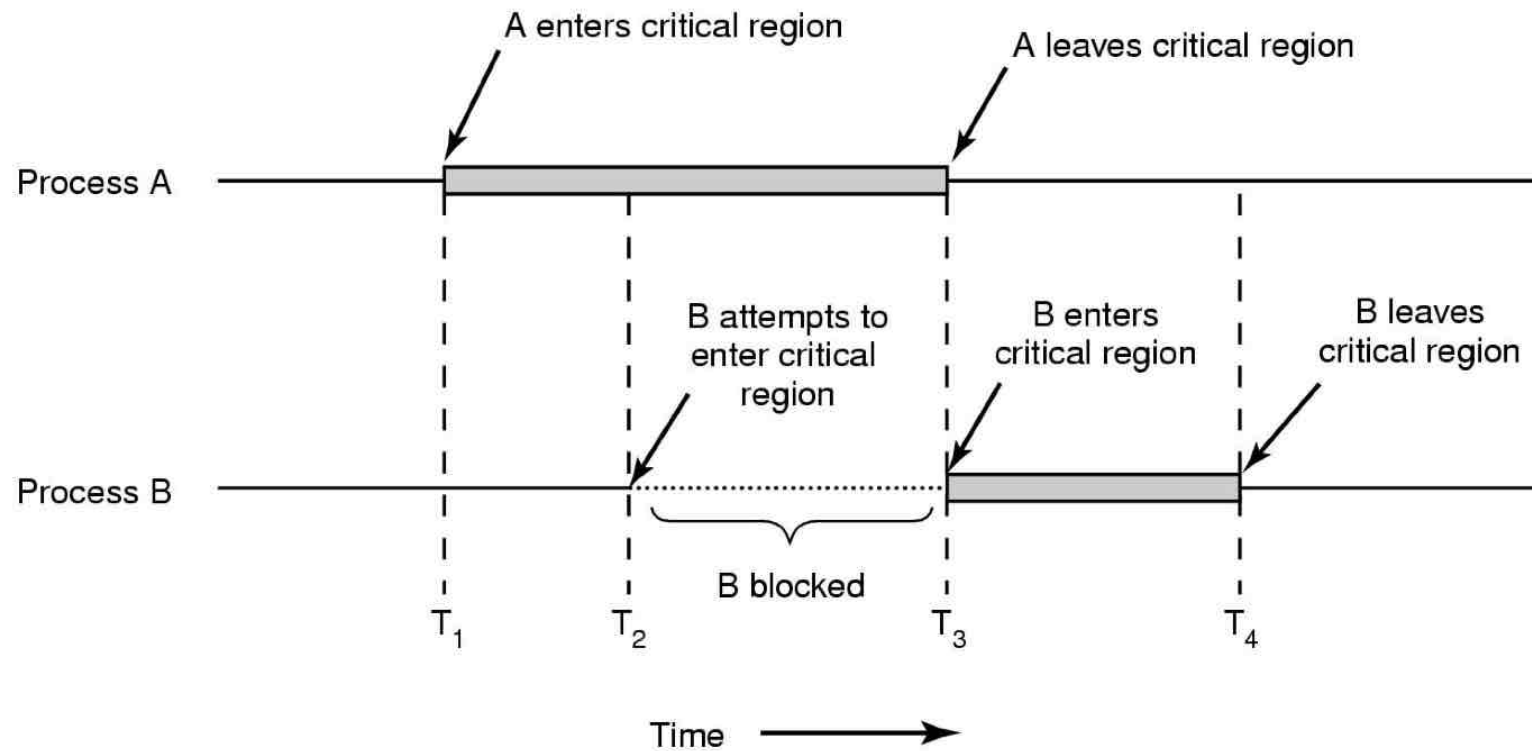
- **Race condition** occurs when two or more processes are reading or writing some shared data and the final result depends on who runs precisely when.



How to Avoid Races

- Mutual exclusion – only one process at a time can use a shared variable/file
- Critical regions – that part of the program where the shared memory is accessed
- Solution – Ensure that two processes can't be in the critical region at the same time

What we are trying to do



The Producer-Consumer Problem (aka Bounded Buffer Problem)

```
#define N 100                                     /* number of slots in the buffer */
int count = 0;                                    /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {                                /* repeat forever */
        item = produce_item();                    /* generate next item */
        if (count == N) sleep();                  /* if buffer is full, go to sleep */
        insert_item(item);                        /* put item in buffer */
        count = count + 1;                        /* increment count of items in buffer */
        if (count == 1) wakeup(consumer);        /* was buffer empty? */
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {                                /* repeat forever */
        if (count == 0) sleep();                  /* if buffer is empty, got to sleep */
        item = remove_item();                    /* take item out of buffer */
        count = count - 1;                        /* decrement count of items in buffer */
        if (count == N - 1) wakeup(producer);    /* was buffer full? */
        consume_item(item);                      /* print item */
    }
}
```

The problem with sleep and wake-up calls

- Empty buffer, $\text{count} == 0$
- Consumer gets replaced by producer before it goes to sleep
- Produces something, $\text{count}++$, sends wakeup to consumer
- Consumer not asleep, ignores wakeup, thinks $\text{count} = 0$, goes to sleep
- Producer fills buffer, goes to sleep
- P and C sleep forever
- So the problem is lost wake-up calls

Semaphores

- Semaphore is an integer variable
- Used to sleeping processes/wakeups
- Two operations, **down** and **up**
- Down checks semaphore. If not zero, decrements semaphore. If zero, process goes to sleep
- Up increments semaphore. If more than one process asleep, one is chosen randomly and enters critical region (first does a down)

Producer Consumer with Semaphores

- 3 semaphores: full, empty and mutex
- Full counts full slots (initially 0)
- Empty counts empty slots (initially N)
- Mutex protects variable which contains the items produced and consumed

Producer Consumer with semaphores

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}

. . .
```

/ number of slots in the buffer */*
/ semaphores are a special kind of int */*
/ controls access to critical region */*
/ counts empty buffer slots */*
/ counts full buffer slots */*

/ TRUE is the constant 1 */*
/ generate something to put in buffer */*
/ decrement empty count */*
/ enter critical region */*
/ put new item in buffer */*
/ leave critical region */*
/ increment count of full slots */*

/ infinite loop */*
/ decrement full count */*
/ enter critical region */*
/ take item from buffer */*
/ leave critical region */*
/ increment count of empty slots */*
/ do something with the item */*

Mutexes

- Don't always need counting operation of semaphore, just mutual exclusion part
- Mutex: variable which can be in one of two states-locked (0), unlocked(1 or other value)
 - Easy to implement

Condition Variables

- Allows a process/thread to block if a condition is not met, e.g. Producer-Consumer. Producer needs to block if the buffer is full.
- Condition variable makes it possible to put producer to sleep if buffer is full

Monitors

- Easy to make a mess of things using mutexes and condition variables. Little errors cause disasters.
- Producer consumer with semaphores – interchange two downs in producer code causes deadlock
- Monitor is a **language construct** which enforces mutual exclusion and blocking mechanism
- C does not have monitor

Monitors (Cont'd)

- Monitor consists of {procedures, data structures, and variables} grouped together in a “module”
- A process can call procedures inside the monitor, but cannot directly access the stuff inside the monitor
- C does not have monitors

Monitor-a picture

```
monitor example
    integer i;
    condition c;

    procedure producer( );
    .
    .
    end;

    procedure consumer( );
    . . .
    end;
end monitor;
```

Producer Consumer Monitor

```
monitor ProducerConsumer
  condition full, empty;
  integer count;

  procedure insert(item: integer);
  begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty)
  end;

  function remove: integer;
  begin
    if count = 0 then wait(empty);
    remove = remove_item;
    count := count - 1;
    if count = N - 1 then signal(full)
  end;

  count := 0;
end monitor;

procedure producer;
begin
  while true do
  begin
    item = produce_item;
    ProducerConsumer.insert(item)
  end
end;

procedure consumer;
begin
  while true do
  begin
    item = ProducerConsumer.remove;
    consume_item(item)
  end
end;
```

Monitors: Good vs Bad

- The good – No messy direct programmer control of semaphores
- The bad – You need a language which supports monitors (Java).
- OS's are written in C

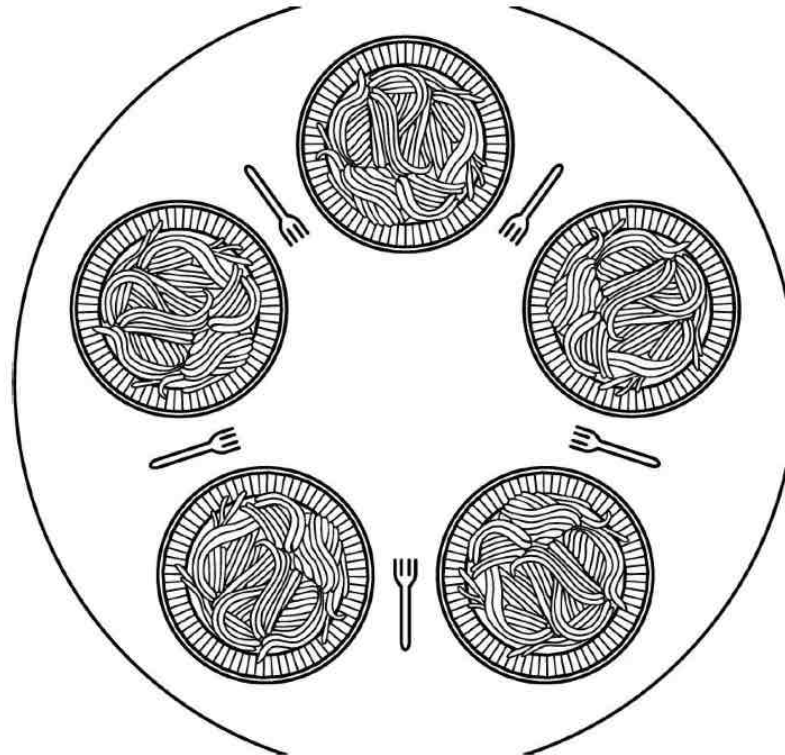
Semaphores: Good vs Bad

- The good – Easy to implement
- The bad – Easy to mess up

Reality

- Monitors and semaphores only work for shared memory
- Don't work for multiple CPU's which have their own private memory, e.g. workstations on an Ethernet

Dining Philosophers Problem



. Lunch time in the Philosophy Department.



Dining Philosophers Problem

```
#define N 5                                /*number of philosophers*/
Void philosopher(int i)                    /*i:philosopher number, from 0 to 4*/
{
While(TRUE){
    think();                              /*philosopher is thinking*/
    take_fork(i);                          /*take left fork*/
    take_fork(i+1)%N;                      /*take right for;% is modulo operator*/
    eat();                                 /*self-explanatory*/
    put_fork(i);                           /*put left fork back on table*/
    put_fork(i+1)%N;                       /*put right fork back on table*/
}
}
```

A **nonsolution** to the dining philosophers problem.



Dining Philosophers Problem

```
#define N          5                /* number of philosophers */
#define LEFT      (i+N-1)%N        /* number of i's left neighbor */
#define RIGHT     (i+1)%N          /* number of i's right neighbor */
#define THINKING  0                /* philosopher is thinking */
#define HUNGRY    1                /* philosopher is trying to get forks */
#define EATING    2                /* philosopher is eating */
typedef int semaphore;             /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)           /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {                /* repeat forever */
        think();                  /* philosopher is thinking */
        take_forks(i);            /* acquire two forks or block */
        eat();                    /* yum-yum, spaghetti */
        put_forks(i);             /* put both forks back on table */
    }
}
```

A solution to the dining philosophers problem. $N=5$.



Dining Philosophers Problem

```
    . . .  
void take_forks(int i)                /* i: philosopher number, from 0 to N-1 */  
{  
    down(&mutex);                     /* enter critical region */  
    state[i] = HUNGRY;                /* record fact that philosopher i is hungry */  
    test(i);                          /* try to acquire 2 forks */  
    up(&mutex);                       /* exit critical region */  
    down(&s[i]);                      /* block if forks were not acquired */  
}  
  
    . . .
```

. A solution to the dining philosophers problem.



Dining Philosophers Problem (5)

• • •

```
void put_forks(i)                                /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                                /* enter critical region */
    state[i] = THINKING;                          /* philosopher has finished eating */
    test(LEFT);                                  /* see if left neighbor can now eat */
    test(RIGHT);                                 /* see if right neighbor can now eat */
    up(&mutex);                                   /* exit critical region */
}

void test(i) /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

A solution to the dining philosophers problem.



Thank You